# PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

# FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 2
3ra práctica (tipo b)
(Primer Semestre 2026)

Indicaciones Generales:
Duración: **110 minutos.**

- No se permite el uso de apuntes de clase, fotocopias ni material impreso.
- No se pueden emplear variables globales, ni objetos (con excepción de los elementos de **iostream**, **iomanip** y **fstream**). No se puede utilizar la clase **string**. Tampoco se podrán emplear las funciones de C que gesten memoria como **malloc**, **realloc**, **memset**, **strdup**, **strtok** o similares, igualmente no se puede emplear cualquier función contenida en las bibliotecas **stdio.h**, **cstdio** o similares y que puedan estar también definidas en otras bibliotecas. No podrá emplear plantillas en este laboratorio.
- Deberá modular correctamente el proyecto en archivos independientes. Las soluciones deberán desarrollarse bajo un estricto diseño descendente. Cada función no debe sobrepasar las 20 líneas de código aproximadamente. El archivo main.cpp solo podrá contener la función main de cada proyecto y el código contenido en él solo podrá estar conformado por tareas implementadas como funciones. En el archivo main.cpp deberá incluir un comentario en el que coloque claramente su nombre y código, de no hacerlo se le descontarán 0.5 puntos en la nota final.
- El código comentado no se calificará. De igual manera no se calificará el código de una función si esta función no es llamada en ninguna parte del proyecto o su llamado está comentado.
- Los programas que presenten errores de sintaxis o de concepto se calificarán en base al 40% de puntaje de la pregunta. Los que no muestren resultados o que estos no sean coherentes en base al 60%.
- Se tomará en cuenta en la calificación el uso de comentarios relevantes.
- Se les recuerda que, de acuerdo con el reglamento disciplinario de nuestra institución, constituye una falta grave copiar del trabajo realizado por otra persona o cometer plagio.
- No se harán excepciones ante cualquier trasgresión de las indicaciones dadas en la prueba.

Puntaje total: 20 puntos

---

- La unidad de trabajo será t:\ (Si lo coloca en otra unidad, no se calificará su laboratorio y se le asignará como nota cero). En la unidad de trabajo t:\ colocará el(los) proyecto(s) solicitado(s).
- Cree allí una carpeta con el nombre Lab03_2026_1_CO_PA_PN donde: CO indica: Código del alumno, PA indica: Primer Apellido del alumno y PN indica: primer nombre. De no colocar este requerimiento se le descontarán 3 puntos de la nota final.

## Cuestionario

- La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en el capítulo 2 del curso: Arreglos y punteros. En este laboratorio se trabajará con punteros genéricos, registros de registros y métodos de asignación de memoria.
- Al finalizar la práctica, comprima la carpeta dada en las indicaciones iniciales empleando el programa Zip que viene por defecto en el Windows, no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares.

Para el diseño de su solución, considere que:

- No podrá emplear arreglos estáticos de más de una dimensión.
- No puede manipular un puntero con más de un índice.
- No puede emplear arreglos auxiliares, estáticos o dinámicos, para guardar los datos de los archivos. Los archivos solo se pueden leer una vez.
- **Puede elegir entre el uso de asignación exacta o dinámica de memoria según crea conveniente.**

## Descripción del caso

Un hospital de tercer nivel requiere un programa en C++ para gestionar su sistema de urgencias clínicas de forma ordenada y reproducible. Actualmente existen varios problemas operativos: los registros de atenciones están fragmentados y no vinculados claramente a los pacientes; no hay un seguimiento consolidado del número de atenciones ni del gasto asociado por paciente; y la información no se presenta en formatos fácilmente legibles para análisis administrativo y clínico.

El sistema propuesto debe permitir, como mínimo:

- Mantener el padrón de pacientes con la información demográfica básica (identificador, nombre, edad y género).
- Registrar las visitas de cada paciente, donde cada visita incluye fecha, hora y el costo de la atención.
- Acumular, en el perfil de cada paciente, el total gastado por todas sus atenciones (campo acumulador costo total).
- Generar un reporte claro, principalmente un resumen por paciente que muestren número de visitas y total gastado, para facilitar la toma de decisiones.

El sistema trabaja con dos archivos CSV de entrada. Cada uno representa una fuente distinta de información y debe ser leído una sola vez durante la ejecución del programa.

**pacientes.csv**

Contiene el padrón de pacientes atendidos por el sistema de urgencias. Cada línea representa un paciente distinto.

| ID | Nombre | Edad | Género |
|---|---|---|---|
| 30001001 | H. Glasspool | 69 | M |
| 30001002 | X. Methuen | 4 | M |
| 30001003 | P. Schubuser | 56 | F |

**visitas.csv**

Contiene el historial de atenciones médicas. Cada línea registra una atención a un paciente en una fecha y hora determinadas y el costo de la visita. Si varias líneas comparten la misma fecha, hora e id del paciente, pertenecen a una misma visita.

| Fecha | Hora | Id Paciente | Costo Atención |
|---|---|---|---|
| 2026-04-01 | 08:00 | 30001001 | 75.00 |
| 2026-04-01 | 08:00 | 30001001 | 98.00 |
| 2026-04-02 | 08:31 | 30001004 | 94.00 |
| 2026-04-02 | 08:31 | 30001004 | 117.00 |

Con esta información se solicita elaborar un proyecto en CLion cuya función “main” estará compuesta por el siguiente código:

```
int main() {
    void* pacientes;
    cargarPacientes("pacientes.csv", pacientes);
    cargarVisitas("visitas.csv", pacientes);
    generarReporte("reporte.txt", pacientes);

    return 0;
}
```

## Parte 1: Carga de Datos Iniciales (6 puntos)

Implemente la función **cargarPacientes**, la cual se encargará de realizar la carga inicial de la estructura **pacientes**, leyendo los datos del padrón de pacientes desde el archivo pacientes.csv, asignando memoria dinámica.

La función debe construir la estructura de datos inicial leyendo el archivo **pacientes.csv**. Para cada paciente se debe almacenar: identificador del paciente, nombre completo, edad y género. La estructura inicial de datos **pacientes** a la que hace referencia esta función está ilustrada en la Figura 1.

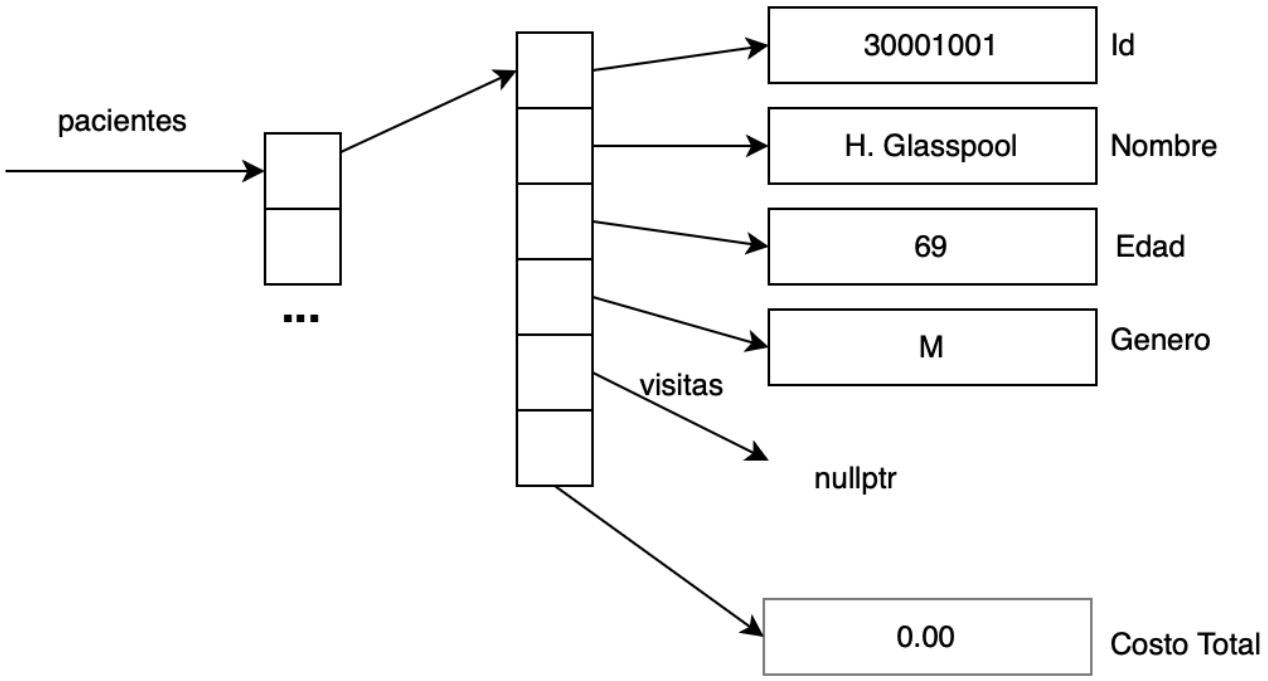


*Figura 1 — Estructura Inicial*

En la estructura **pacientes**, el campo correspondiente al arreglo de visitas debe ser inicializado en nullptr durante esta fase del laboratorio, dado que las visitas se cargarán en la Parte 2. Los campos id, nombre, edad y género deben cargarse directamente desde el archivo CSV. En cuanto al campo total gastado, que acumula el monto costo total asumido por el paciente en todas sus atenciones debe ser inicializado en 0.0 en esta etapa, ya que su actualización se realizará durante la Parte 2.

## Parte 2 (10 puntos)

Es importante destacar que esta parte depende directamente de la correcta implementación de la Parte 1. Por lo tanto, asegúrese de que la carga inicial de la estructura de pacientes esté completa antes de continuar.

Implemente la función **cargarVisitas**, la cual recibirá como parámetros la estructura de pacientes previamente cargada y la ruta al archivo visitas.csv, y utilizará ese archivo para registrar las atenciones realizadas a cada paciente.

La función **cargarVisitas** creará una estructura dinámica de visitas utilizando asignación de memoria dinámica, la cual será referenciada por el campo **visitas** de cada paciente, como se muestra en la Figura 2. La estructura de cada visita estará compuesta por la fecha y hora de atención y el costo de la atención.

Para realizar la vinculación entre visitas y pacientes, la función deberá buscar al paciente correspondiente a cada línea del archivo visitas.csv a través de su identificador. La búsqueda deberá realizarse recorriendo manualmente el arreglo de pacientes; no se podrá utilizar la función bsearch.

Si el identificador del paciente de una línea no existe en el sistema, dicha línea deberá descartarse.

Durante este proceso, se actualizará el campo Costo Total de cada paciente, el cual contendrá la suma del costo de todas las atenciones de cada paciente.

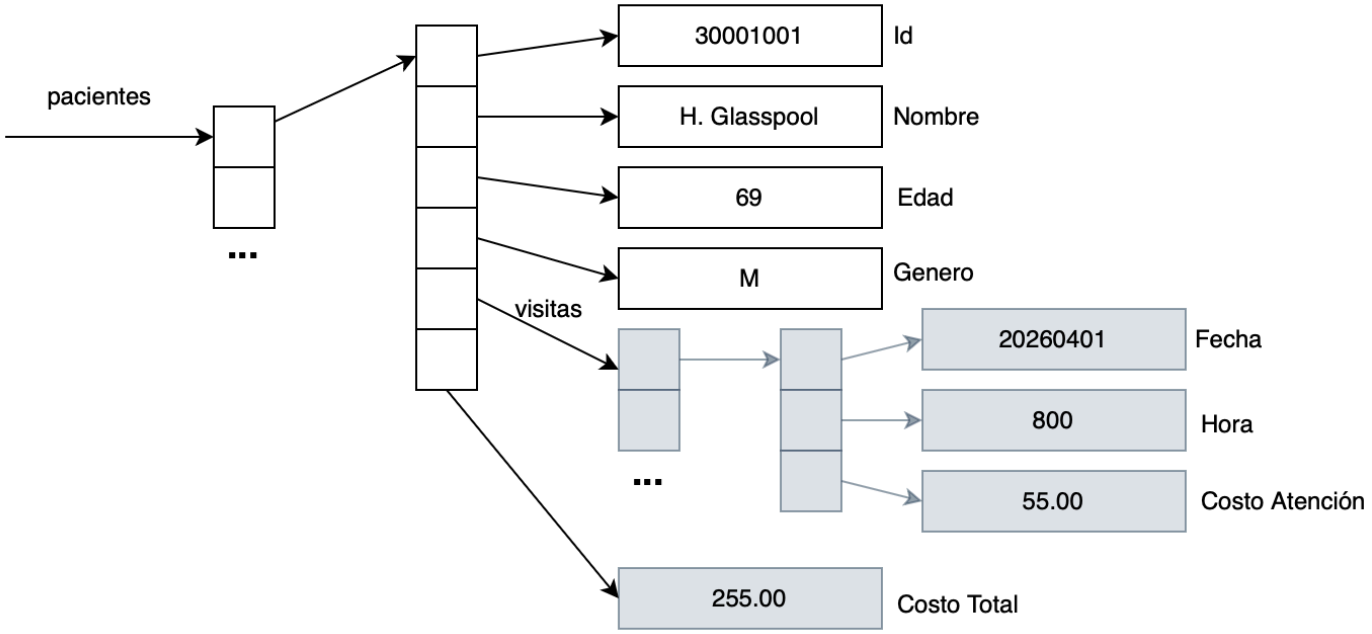


*Figura 2 — Estructura con visitas actualizadas*

## Parte 3 (4 puntos)

Es importante destacar que esta sección depende directamente de la correcta implementación de las dos partes anteriores. Por lo tanto, asegúrese de que todas las etapas previas estén completadas antes de continuar.

Implemente la función **generarReporte**, la cual recibirá como parámetro el nombre del archivo de salida y la estructura de pacientes, y generará un informe que deberá ser guardado en el archivo reporte.txt.

El reporte solo incluirá información de los pacientes que tengan al menos una visita registrada, y mostrará los datos del paciente y el costo total asumido por el paciente por todas sus atenciones.

El formato del archivo de salida es el siguiente:

```
==========================================================================================
                                REPORTE DEL SISTEMA DE URGENCIAS
==========================================================================================
------------------------------------------------------------------------------------------
ID          Nombre          Edad        Genero      Visitas                 Total (S/)
------------------------------------------------------------------------------------------
30001001    H. Glasspool    69          M           3                       294.00
30001002    X. Methuen      4           M           3                       351.00
30001003    P. Schubuser    56          F           3                       408.00
```

Profesores del curso: Ana Roncal, Miguel Guanira, Rony Cueva, Erasmo Gómez, Eric Huiza

Pando, 24 de abril de 2025